version of the base Identity context in this case and providing the `ApplicationUser` type.

Listing 23.5 Updating `AppDbContext` to use `IdentityDbContext`

```
public class AppDbContext : IdentityDbContext<ApplicationUser>    ❶
{
    public AppDbContext(DbContextOptions<AppDbContext> options)   ❷
        : base(options)                                           ❷
    { }                                                           ❷

    public DbSet<Recipe> Recipes { get; set; }                    ❷
}
```

❶ Updates to inherit from the Identity context instead of directly from DbContext

❷ The remainder of the class remains the same.

Effectively, by updating the base class of your context in this way, you've added a whole load of new entities to EF Core's data model. As you saw in chapter 12, whenever EF Core's data model changes, you need to create a new migration and apply those changes to the database.

At this point, your app should compile, so you can add a new migration called `AddIdentitySchema` using

```
dotnet ef migrations add AddIdentitySchema
```

The final step is updating your application's Razor Pages and layouts to reference the default identity UI. Normally, adding 30 new Razor Pages to your application would be a lot of work, but using the default Identity UI makes it a breeze.

23.4.3 Updating the Razor views to link to the Identity UI

Technically, you don't have to update your Razor Pages to reference the pages included in the default UI, but you probably want to add the login widget to your